

UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

INGENIERÍA DE SISTEMAS COMPUTACIONALES

LICENCIATURA EN DESARROLLO Y GESTIÓN DE SOFTWARE

Informe de Actividad en Clase N.º 3

Nodo de telemetría del equipo

Nombre	Querube Ariza
Grupo	1GS134
Asignatura	Desarrollo de Software VIII
Profesor	Kexy Rodríguez
Tema	Módulos, paquetes y programación orientada a eventos
Fecha	30/09/2026

Proyecto: monitor_iot_QuerubeAriza

Dashboard personalizado: laptop-QuerubeAriza

Tipografía del informe: Arial 12 pt. Figuras y tablas numeradas con descripción; páginas numeradas.

Índice

1. Objetivo	3
2. Introducción	4
3. Estructura del proyecto	5
4. Personalización del panel de control	6
5. Sistema de alertas sonoras	7
5.1 Tabla de alertas sonoras diseñadas	7
5.2 Criterios de diseño del conjunto	10
5.3 Cómo se evitó que el sonido bloquee el ciclo de monitoreo	10
6. Ejecución del código y capturas de pantalla	13
6.1 Revisión del entorno	13
6.2 Dashboard de consola en ejecución	13
6.3 Servidor del dashboard web	14
6.4 Ventana gráfica (tkinter)	14
6.5 Dashboard web en el navegador	15
7. Eventos detectados durante la ejecución	17
7.1 Eventos por umbral, histéresis y flanco	17
7.2 Evento por falla o ausencia	17
7.3 Reporte por tiempo y bitácora JSON	18
8. Pruebas unitarias	19
8.1 Resultado de la ejecución	19
8.2 Capturas de las pruebas unitarias	22
9. Conclusión	24

1. Objetivo

Construir un nodo de telemetría del equipo en Python, organizado con módulos y paquetes y con una arquitectura orientada a eventos, que mida en tiempo real el uso del procesador, la memoria RAM, el disco, la red, los procesos y la batería; que detecte condiciones de alarma; y que presente la información en tres tableros: consola, ventana gráfica y web local.

Objetivos específicos:

1. Separar responsabilidades en paquetes (sensores, eventos, almacenamiento, dashboard) para que cada uno tenga una sola razón de cambio.
2. Aplicar programación orientada a eventos: detección por umbral con histéresis, por flanco, por tiempo y por falla o ausencia de un sensor, despachada mediante un diccionario de manejadores.
3. Implementar un subsistema de alertas sonoras (sonidos.py) que sintetice los WAV en memoria y los reproduzca sin bloquear el ciclo de monitoreo.
4. Diseñar y documentar la tabla de alertas sonoras con la justificación de cada sonido y explicar cómo se evitó que el audio detuviera el monitoreo.
5. Ejecutar el código, capturar pantalla de cada ejecución, correr las pruebas unitarias y presentar sus resultados y conclusiones en este informe con figuras enumeradas y páginas numeradas.

2. Introducción

Este informe documenta el desarrollo de un nodo de telemetría del equipo, implementado en Python mediante módulos y paquetes. El sistema obtiene mediciones reales del computador usando la biblioteca psutil: uso del procesador, memoria RAM, disco, red, procesos y batería.

La práctica aplica programación orientada a eventos. El núcleo no utiliza esperas bloqueantes con `time.sleep()` para decidir cuándo leer sensores; en su lugar conserva marcas de tiempo y ejecuta cada grupo de lecturas cuando corresponde. Los eventos son detectados por umbral con histéresis, por flanco, por tiempo y por falla o ausencia de un sensor.

El proyecto separa responsabilidades: los sensores solo leen, almacenamiento guarda las ventanas móviles y la bitácora JSON, detectores decide si ocurrió un evento, manejadores reaccionan, y el dashboard presenta la información. Además del dashboard gráfico en tkinter y la consola, se agregó una versión web local para ejecutar el panel en el navegador.

Al sistema se le añadió un laboratorio de alertas sonoras: 21 eventos disponen de un patrón sonoro propio, sintetizado como archivo WAV en memoria y emitido por un hilo independiente. El requisito central de ese laboratorio es que, mientras suena una alerta, el tablero siga refrescando y el ciclo siga detectando eventos; la sección 5 explica el diseño, la justificación de cada sonido y las pruebas que lo verifican.

3. Estructura del proyecto

La estructura del proyecto se muestra a continuación. Cada paquete responde por una sola responsabilidad y el punto de entrada main.py solo elige la presentación, sin contener umbrales ni tiempos.

```
monitor_iot_QuerubeAriza/
|-- Informe_Nodo_Telemetria_Querube_Ariza.pdf
|-- almacenamiento/
|   |-- __init__.py
|   |-- registro.py
|-- bitacora.json
|-- config.py
|-- dashboard/
|   |-- __init__.py
|   |-- consola.py
|   |-- ventana.py
|   |-- web.py
|-- eventos/
|   |-- __init__.py
|   |-- conexion_red.py
|   |-- despachador.py
|   |-- detectores.py
|   |-- manejadores.py
|-- generador.py
|-- generar_informe.py
|-- main.py
|-- nucleo.py
|-- pruebas/
|   |-- __init__.py
|   |-- en_vivo.py
|   |-- test_conexion_red.py
|   |-- test_integracion.py
|   |-- test_sonidos.py
|   |-- test_ventana.py
|-- requirements.txt
|-- sensores/
|   |-- __init__.py
|   |-- bateria.py
|   |-- cpu.py
|   |-- disco.py
|   |-- memoria.py
|   |-- procesos.py
|   |-- red.py
|-- sonidos.py
```

Figura 1. Estructura de carpetas y paquetes del proyecto monitor_iot_QuerubeAriza.

Se observan los paquetes sensores (un módulo por métrica), eventos (detectores, manejadores y despachador), almacenamiento (ventana móvil y bitácara) y dashboard (consola, ventana gráfica y web), además de config.py con toda la parametrización, nucleo.py con el bucle orientado a eventos, sonidos.py con el reproductor de alertas, generador.py para provocar carga y el paquete pruebas con las pruebas unitarias.

4. Personalización del panel de control

El panel de control fue personalizado con los datos del estudiante. En config.py se configuró el nodo como laptop-QuerubeAriza y la ubicación como Querube Ariza - Laboratorio 3 FISC. Estos valores aparecen en el encabezado del dashboard de consola, de tkinter y de la versión web.

```
=====
NODO laptop-QuerubeAriza | Querube Ariza - Laboratorio 3 FISC | 14:38:55
=====
Uso de CPU      [##### - - - - -] 43.1 %
                 8 nucleos | 987 MHz
Memoria RAM     [##### - - - - -] 68.7 %
                 10.8 de 15.7 GB | swap 3%
Disco           [##### - - - - -] 60.4 %
                 188.3 GB libres de 475.5 GB
Red             [ - - - - -] 3.5 KB/s
                 sube 0.0 | baja 3.5 KB/s | total 937 MB
Procesos        [##### - - - - -] 273 procs
                 sin proceso dominante
Bateria         [##### - - - - -] 23 %
                 en bateria | quedan 0h 17m
-----
Ultimos eventos:
... salida recortada para el informe ...
Memoria RAM     [##### - - - - -] 68.8 %
                 10.8 de 15.7 GB | swap 3%
Disco           [##### - - - - -] 60.4 %
                 188.3 GB libres de 475.5 GB
Red             [ - - - - -] 3.2 KB/s
                 sube 0.0 | baja 3.2 KB/s | total 937 MB
Procesos        [##### - - - - -] 273 procs
                 sin proceso dominante
Bateria         [##### - - - - -] 23 %
                 en bateria | quedan 0h 17m
-----
Ultimos eventos:
 14:38:54 [INFO ] Nodo laptop-QuerubeAriza iniciado | sensores activos
 14:38:54 [AVISO] El sistema empezo a usar memoria de intercambio
-----
Ctrl+C para terminar
```

Figura 2. Tablero de consola personalizado con el nombre del nodo y la ubicación.

Captura del dashboard de consola después de la personalización: la primera línea identifica el nodo laptop-QuerubeAriza y su ubicación, y debajo aparecen las seis métricas con barra de progreso, porcentaje, la lista de los últimos eventos y la leyenda de teclas. El mismo encabezado se reutiliza en la ventana gráfica y en la versión web.

5. Sistema de alertas sonoras

El subsistema de alertas sonoras vive en `sonidos.py` y se parametriza por completo en `config.py`: allí están definidos los 17 patrones (`SONIDO_PATRONES`) y la asignación de un patrón a cada uno de los 21 eventos (`SONIDO_EVENTOS`). Cada nota indica tipo de onda (seno, cuadrada, triangular, barrido, ruido o silencio), duración en segundos y, según corresponda, frecuencia, volumen o frecuencia inicial y final.

El reproductor sintetiza cada patrón en un WAV de 22 050 muestras por segundo, mono y de 16 bits con signo, directamente en memoria, y lo pasa a `winsound`. Un fundido de 8 ms al inicio y al final de cada nota evita el clic al cambiar de tono, y el volumen maestro está limitado al 30 % para que la alerta sea audible sin resultar molesta.

5.1 Tabla de alertas sonoras diseñadas

La tabla 1 reúne los sonidos de problema y de falla; la tabla 2 reúne los sonidos de recuperación e informe. La última columna de ambas contiene la justificación del sonido: por qué esa forma de onda y esa dirección de frecuencia se eligieron para ese evento.

Tabla 1. Alertas sonoras de problema y de falla.

Catorce eventos de estado crítico. Cada fila indica el evento, el patrón asignado, la composición exacta de las notas y la justificación acústica de la elección.

Evento	Patrón	Composición del sonido	Justificación del sonido
conexion_perdida	sirena_caída	Barrido 880 a 160 Hz (0,9 s), pausa, barrido 660 a 120 Hz (0,7 s)	El tono que cae se interpreta como algo que se apaga: es el aviso mas urgente del sistema y por eso suena con el barrido mas largo y descendente.
conexion_recordatorio	latido_caída	Dos golpes cuadrados de 220 Hz y uno de 146,83 Hz, separados por silencio	Un latido grave y repetido recuerda que la caída sigue sin resolverse, sin la urgencia de la sirena inicial, para no saturar al operador.
cpu_alta	doble_tic	Dos tics de seno de 1567,98 Hz de 0,07 s con silencio intermedio	Tics cortos y agudos, tipo semaforo: informan de un pico de trabajo sin convertirse en una alarma continua que taparia los demas avisos.
nucleo_saturado	tic_nucleo	Dos notas de seno de 987,77 Hz y 1318,51 Hz de 0,06 s	Variacion corta y media del doble tic de CPU: el oído la asocia al mismo familia de problema, pero a un nivel menor (un solo nucleo).
ram_alta	acorde_memoria	Tres notas 440 / 587,33 / 739,99 Hz y repetida 739,99 Hz al 80 %	Acorde ascendente que se queda tenso: la memoria es un recurso que permanece ocupado, por eso el sonido no cierra, sino que se sostiene.

Evento	Patrón	Composición del sonido	Justificación del sonido
disco_lleno	golpe_grave	Cuadrada de 110 Hz (0,28 s) seguida de seno de 98 Hz (0,3 s)	Golpe seco y grave: da la sensación de lleneo y peso, coherente con un disco que ya no tiene espacio para recibir información.
red_pico	swoosh_subida	Barrido de 320 a 1760 Hz (0,4 s) al 60 % de volumen	Barrido ascendente: codifica el tráfico que sube de golpe; el movimiento creciente del tono se asocia con un flujo que aumenta.
bateria_baja	pip_bateria	Tres pitidos de 440 Hz de 0,22 s separados por 0,3 s de silencio	Pitidos lentos e iguales: suenan como una cuenta regresiva de energía que se agota y exigen buscar el cargador.
cargador_desconectado	plano_descarga	Barrido de 700 a 300 Hz (0,25 s) y cuadrada de 300 Hz (0,2 s)	El tono se va al piso y termina plano: traduce literalmente la energía que se corta al desconectar el equipo.
swap_activo	pulso_swap	Dos pulsos cuadrados de 261,63 Hz separados por 0,1 s de silencio	Pulso binario de dos tiempos: refleja el vaiven de la memoria entre RAM y disco que ocurre mientras el swap está activo.
sensor_ausente	zumbido_falla	Cuadrada de 133 Hz, ruido blanco de 0,12 s y otra cuadrada de 133 Hz	Zumbido aspero con ruido: suena deliberadamente roto para distinguir una falla de hardware de un simple aviso de estado.
lectura_invalida	tictac_invalido	Tres tics de seno de 1174,66 Hz de 0,06 s con silencios intermedios	Tics nerviosos y rápidos: transmiten una lectura dudosa, corta e inestable que no llega a confirmarse.
salto_anomalo	zigzag_salto	Barrido 1400 a 300 Hz y vuelta inmediata 300 a 1400 Hz	Ida y vuelta brusca del tono: reproduce el salto entre dos lecturas muy distintas consecutivas.
proceso_pesado	grunido_proceso	Triangular de 196 Hz (0,2 s) y cuadrada de 196 Hz (0,12 s)	Grave y burdo, casi un grunido: identifica un proceso que traga recursos de forma sostenida.

Tabla 2. Alertas sonoras de recuperación e informe.

Siete eventos que indican que una condición crítica desapareció. Casi todos reutilizan la misma campanita corta para que el operador reconozca de inmediato que no debe intervenir.

Evento	Patrón	Composición del sonido	Justificación del sonido
conexion_recuperada	arreglo_ascendente	Do 523,25 / Mi 659,25 / Sol 783,99 / Do 1046,5 Hz	Arreglo ascendente mayor: convención musical de todo volvió a la normalidad; la última nota sube de volumen para cerrar con claridad.
cpu_normal	chime_corto	Campanita de 1046,5 Hz (0,1 s) y 1318,51 Hz (0,14 s) al 60 %	Campana corta, suave y agradable: es el sonido genérico de alivio y por eso se reutiliza en todas las condiciones normalizadas.

Evento	Patrón	Composición del sonido	Justificación del sonido
ram_normal	chime_corto	Misma campanita corta de dos notas	Al repetir el mismo sonido en todas las recuperaciones, el operador reconoce de inmediato que no hay que intervenir.
disco_aliviado	chime_corto	Misma campanita corta de dos notas	Confirma que el disco volvió bajo el umbral sin añadir un sonido nuevo que hubiera que memorizar.
red_calma	chime_corto	Misma campanita corta de dos notas	El tráfico bajo al nivel de calma: mismo aviso agradable de situación resuelta.
bateria_recuperada	arreglo_ascendente	Mismo arreglo do-mi-sol-do ascendente	La batería dejó la zona de riesgo: se reutiliza el arreglo mayor porque es el sonido más reconocible de recuperación total.
cargador_conectado	chime_corto	Misma campanita corta de dos notas	Aviso breve de que la fuente de energía volvió a estar disponible.

5.2 Criterios de diseño del conjunto

- Dirección del tono: los problemas bajan el tono (sirena de caída, plano de descarga, golpe de disco) y las recuperaciones lo suben (arreglo ascendente, campanita). El oído interpreta esa dirección antes de entender el mensaje.
- Frecuencia según el recurso: los almacenamientos usan frecuencias graves y densas (disco 98-110 Hz, swap 261 Hz), la CPU usa tics agudos (1568 Hz) y la red usa barridos amplios (320-1760 Hz).
- Timbre según la naturaleza del evento: lo que está bien suena a seno limpio; lo que está roto suena a cuadrada y ruido (zumbido de falla); lo que oscila suena a barrido (zigzag de salto anómalo).
- Duración según la urgencia: la sirena de caída dura 1,72 s en total, mientras un tic de CPU dura 0,14 s para no tapar los demás avisos.
- Un solo sonido de alivio: todas las condiciones normalizadas comparten la campanita corta, de modo que no haya que memorizar siete sonidos distintos para el mismo significado.
- Sin patrón, sin sonido: un evento que no esté en SONIDO_EVENTOS se descarta en silencio, lo que permite agregar eventos nuevos sin tocar el reproductor.

5.3 Cómo se evitó que el sonido bloquee el ciclo de monitoreo

El riesgo de una alerta sonora es que el ciclo de monitoreo se detenga mientras suena el tono: `winsound.PlaySound()` es una llamada que espera a que el audio termine, y si se ejecutara dentro de `nucleo.ciclo()` congelaría la lectura de sensores durante cientos de milisegundos. El diseño evita esa condición separando quién detecta de quién suena:

- 1. Quien detecta solo encola.** El manejador del evento llama a `sonidos.reproducir(clave)`, y esa función únicamente agrega la alerta a un deque con su marca de tiempo. No llama a `winsound`, no genera WAV y no espera: devuelve `True` o `False` y regresa al ciclo inmediatamente.
- 2. Quien suena es otro hilo.** `nucleo.iniciar()` llama a `sonidos.iniciar()`, que crea una sola vez un hilo daemon llamado `alertas-sonoras`. Ese hilo tiene su propio bucle: saca la alerta más antigua de la cola, la sintetiza y la reproduce. El ciclo de monitoreo jamás entra en ese bucle ni espera por él.
- 3. La cola es acotada y protegida.** El deque tiene un tope de seis elementos y todos los accesos pasan por un `threading.Lock`. Si el equipo se atrasa, se pierde el aviso más viejo en lugar de acumular una cola infinita de sonidos pendientes.
- 4. Dos controles por marca de tiempo.** La cadencia evita repetir el mismo aviso antes de 1,2 s y el vencimiento descarta alertas de más de 6 s. Así una ráfaga de eventos no se convierte en una ráfaga de sonidos.

5. Ningún fallo de audio detiene el monitoreo. La síntesis y la reproducción están envueltas en try/except dentro del hilo: si no hay tarjeta de audio o el WAV falla, el hilo simplemente sigue con la siguiente alerta y el sistema continúa monitoreando en silencio.

6. La prueba lo demuestra. Las pruebas unitarias comprueban que encolar es inmediato y no llama a la emisión, que encolar no arranca ni toca el hilo, y que las lecturas siguen llegando con el reproductor encendido.

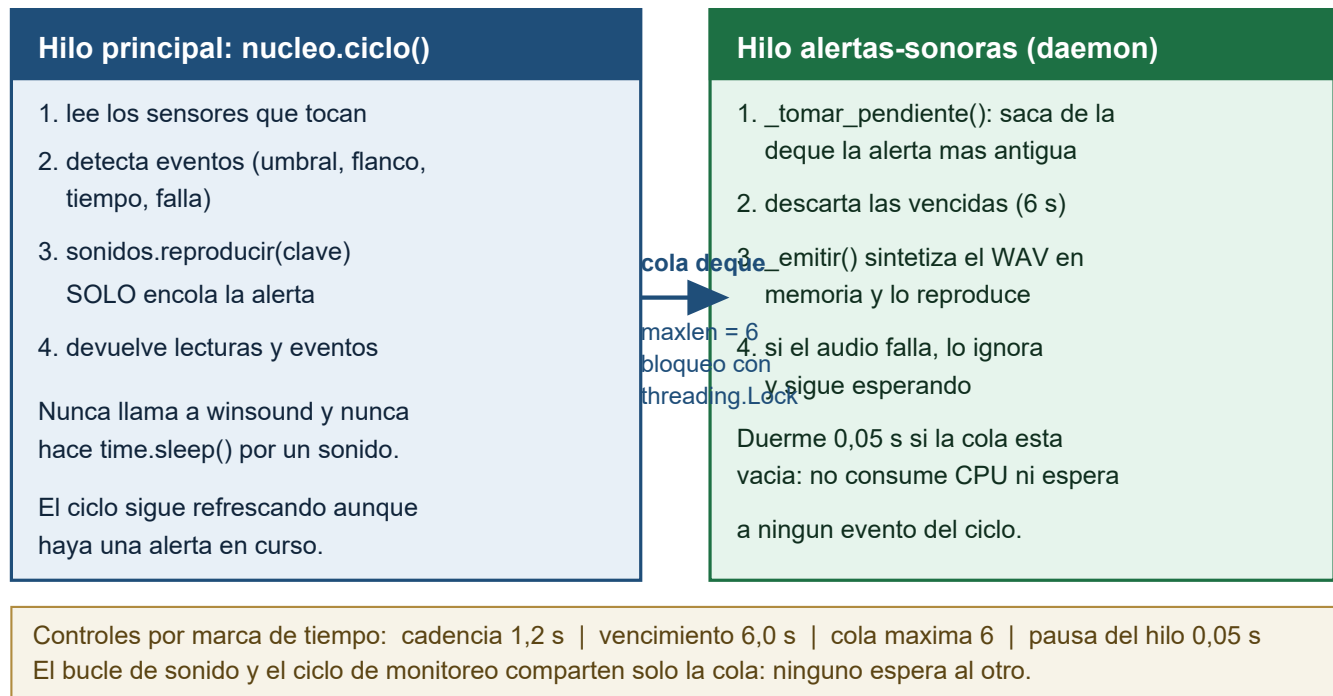


Figura 3. Separación entre el ciclo de monitoreo y el hilo de alertas sonoras.

A la izquierda, el hilo principal: recorre los sensores, detecta los eventos y llama a sonidos.reproducir(), función que solo encola. A la derecha, el hilo alertas-sonoras, que es el único que sintetiza y reproduce el audio. La única zona compartida es la cola deque protegida por un bloqueo; la banda inferior resume los cuatro controles por marca de tiempo que impiden que la cola crezca sin límite.

Tabla 3. Controles por marca de tiempo del reproductor.

Los cuatro valores viven en config.py; ninguno de ellos se define en sonidos.py ni en el ciclo de monitoreo.

Constante	Valor	Qué evita
SONIDO_COOLDOWN	1,2 s	Cadencia: un mismo evento no vuelve a encolarse antes de 1,2 s, por mas veces que se dispare. Evita repetir el mismo tono en cada vuelta rapida.
SONIDO_VENCIDO	6,0 s	Vencimiento: una alerta mas vieja que 6 s se descarta en vez de sonar tarde, de modo que una racha antigua no se reproduce cuando ya no sirve.

Constante	Valor	Qué evita
SONIDO_MAX_COLA	6	Cola acotada: el deque tiene tope de 6 elementos. Si el equipo se atrasa, se pierde el aviso mas viejo y nunca crece una cola infinita de sonidos.
SONIDO_PAUSA	0,05 s	Pausa del hilo: con la cola vacia el reproductor duerme 50 ms en vez de consumir CPU; jamas espera a que el ciclo le entregue un evento.

6. Ejecución del código y capturas de pantalla

Se ejecutaron las distintas formas de arrancar el nodo y se capturó la pantalla de cada una. Cada captura va acompañada de la descripción de lo que en ella se observa.

6.1 Revisión del entorno

```
Revision del entorno
-----
psutil      OK  (version 7.2.2)
tkinter     OK  (Tk 8.6)
-----
Uso de CPU   disponible
Memoria RAM  disponible
Disco        disponible
Red          disponible
Procesos     disponible
Bateria      disponible
-----
La bateria aparece como NO disponible en computadoras de
escritorio. No es un error: es un sensor ausente y el
programa lo maneja como tal.

C:\Users\Windows 11\Downloads\monitor_iot_QuerubeAriza>
```

Figura 4. Ejecución de python main.py --revisar en la terminal.

Se ejecutó python main.py --revisar. La captura muestra el resultado de la revisión: psutil 7.2.2 y tkinter 8.6 están instalados, y debajo aparece la lista de sensores con su estado. Los seis sensores (Uso de CPU, Memoria RAM, Disco, Red, Procesos y Batería) figuran como disponibles, y el mensaje final explica que la batería puede aparecer como NO disponible en computadoras de escritorio, lo cual se maneja como un sensor ausente y no como un error.

6.2 Dashboard de consola en ejecución

```
=====
NODO laptop-QuerubeAriza | Querube Ariza - Laboratorio 3 FISC | 15:40:24
=====
Uso de CPU      [##### - - - - -]      27.2 %
                  8 nucleos | 1382 MHz
Memoria RAM     [##### - - - - -]      77.5 %
                  12.2 de 15.7 GB | swap 8%
Disco           [##### - - - - -]      61.5 %
                  183.0 GB libres de 475.5 GB
Red             [ - - - - -]      24.9 KB/s
                  sube 2.4 | baja 22.4 KB/s | total 1292 MB
Procesos        [##### - - - - -]      288 procs
                  mayor: System Idle Process (623%)
Bateria         [##### - - - - -]      83 %
                  conectado a la corriente
-----
Red (enlace)    conectada (Wi-Fi)
Alertas         sonido activo | cola 0 | ultimo: proceso_pesado
-----
Ultimos eventos:
15:40:09 [INFO ] Nodo laptop-QuerubeAriza iniciado | sensores activos
15:40:09 [AVISO ] El sistema empezo a usar memoria de intercambio
15:40:14 [AVISO ] System Idle Process consume 606.0% de CPU
15:40:19 [AVISO ] System Idle Process consume 609.2% de CPU
15:40:24 [AVISO ] System Idle Process consume 622.9% de CPU
-----
Ctrl+C para terminar
_
```

Figura 5. Ejecución de python main.py --consola con el tablero refrescándose.

Se ejecutó python main.py --consola y se capturó el tablero después de varios refrescos. Se observa el encabezado con el nodo y la hora de cada vuelta, las seis métricas con su barra de progreso y su valor (CPU, RAM, disco, red en KB/s, número de procesos y batería), el bloque de últimos eventos y la leyenda Ctrl+C para terminar. La actualización se hace cada 200 ms sin bloquear el ciclo.

6.3 Servidor del dashboard web

```
Dashboard web corriendo en http://localhost:8000 (Ctrl+C para detener)
_
```

Figura 6. Ejecución de python main.py --web (servidor local).

Se ejecutó python main.py --web. La captura muestra el mensaje del servidor local indicando que el dashboard web quedó corriendo en http://localhost:8000, listo para atender las peticiones del navegador y entregar las lecturas por JSON en el endpoint /datos.

6.4 Ventana gráfica (tkinter)

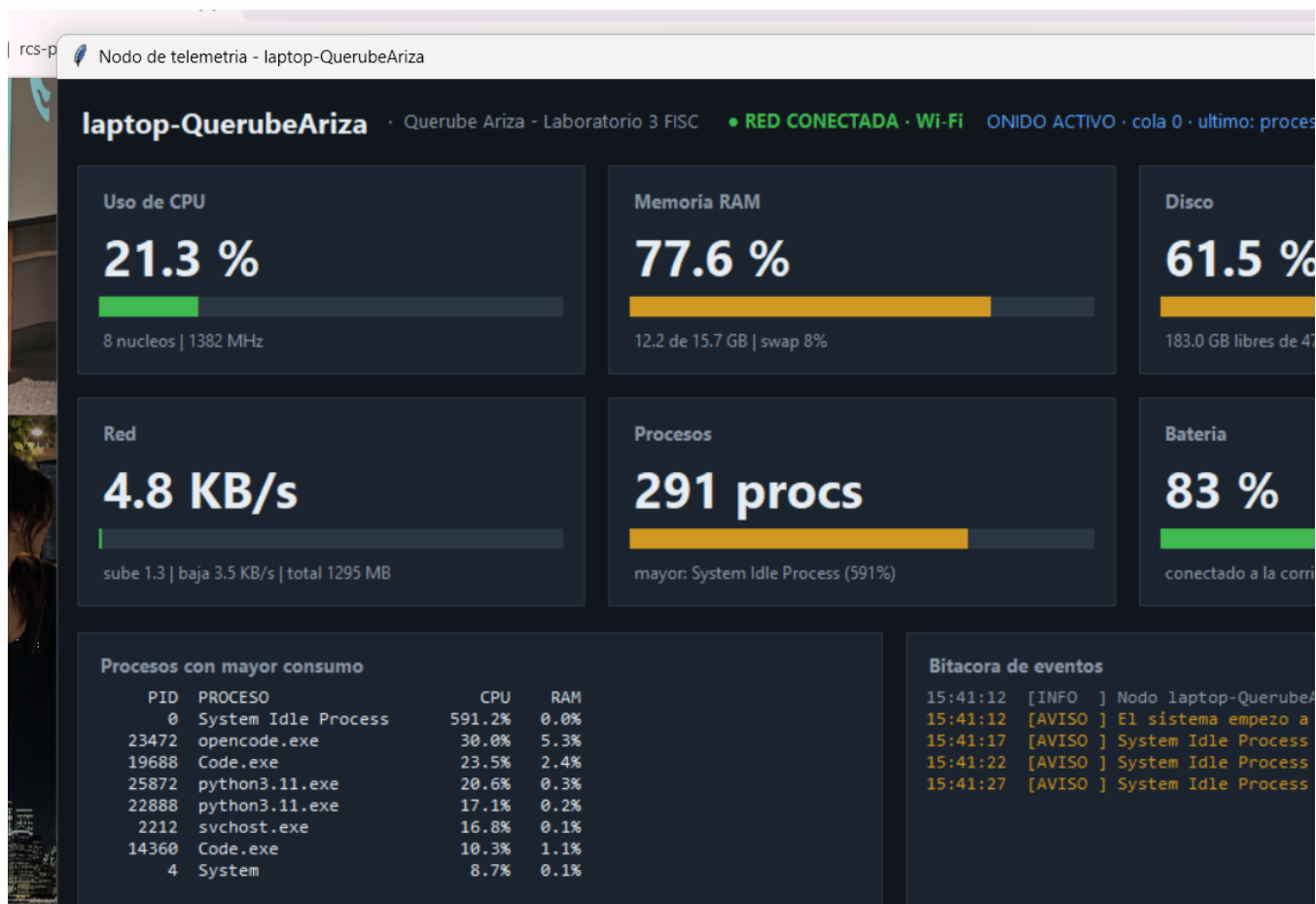


Figura 7. Dashboard gráfico de escritorio (python main.py) en ejecución.

Captura de la ventana gráfica que abre el punto de entrada por defecto. La ventana muestra las seis barras de métricas del nodo, los últimos eventos detectados y los botones de control del tablero, entre ellos el botón que silencia o reactiva las alertas sonoras, coherente con el estado del reproductor descrito en la sección 5.

6.5 Dashboard web en el navegador

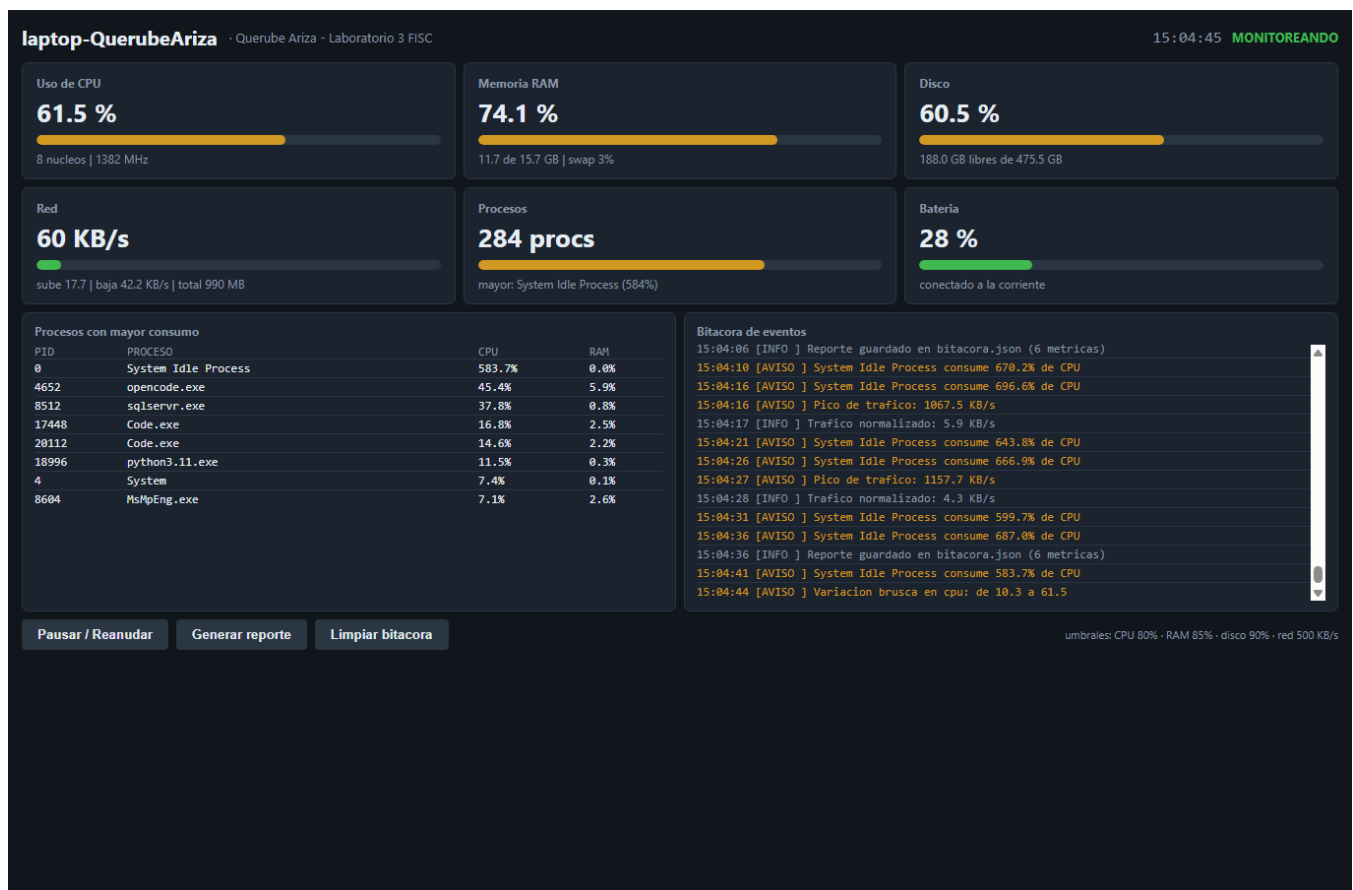


Figura 8. Dashboard web personalizado abierto en el navegador.

Captura del panel web servido por el propio nodo en <http://localhost:8000>. Reproduce las mismas seis métricas y la gráfica del historial, y ofrece los botones Pausar / Reanudar, Generar reporte y Limpiar bitácara. El navegador se refresca cada segundo contra el endpoint /datos, por lo que la interfaz no interfiere con el ciclo de monitoreo.

7. Eventos detectados durante la ejecución

7.1 Eventos por umbral, histéresis y flanco

Con la opción 1 de generador.py se forzó carga de CPU para provocar eventos reales y observar cómo el despachador los atiende.

```
14:44:28 [AVISO ] swap_activo      El sistema empezó a usar memoria de intercambio
14:44:30 [AVISO ] nucleo_saturado  Nucleo 0 saturado (98.4%)
14:44:30 [AVISO ] nucleo_saturado  Nucleo 1 saturado (98.2%)
14:44:30 [AVISO ] nucleo_saturado  Nucleo 2 saturado (97.1%)
14:44:30 [AVISO ] nucleo_saturado  Nucleo 3 saturado (98.1%)
14:44:30 [AVISO ] nucleo_saturado  Nucleo 4 saturado (94.3%)
14:44:30 [AVISO ] nucleo_saturado  Nucleo 5 saturado (97.2%)
14:44:30 [AVISO ] nucleo_saturado  Nucleo 6 saturado (97.2%)
14:44:30 [AVISO ] nucleo_saturado  Nucleo 7 saturado (96.3%)
14:44:31 [ALERTA] cpu_alta      Carga alta sostenida: 85.5% (umbral 80.0%)
14:44:33 [AVISO ] proceso_pesado  System Idle Process consume 152.1% de CPU
14:44:33 [ALERTA] bateria_baja    Bateria baja: 11%
14:44:40 [AVISO ] proceso_pesado  python3.11.exe consume 91.9% de CPU
14:44:45 [AVISO ] proceso_pesado  git.exe consume 66.2% de CPU
14:44:45 [INFO ] nucleo_libre    Nucleo 1 liberado
14:44:45 [INFO ] nucleo_libre    Nucleo 2 liberado
14:44:45 [INFO ] nucleo_libre    Nucleo 5 liberado
14:44:45 [INFO ] nucleo_libre    Nucleo 7 liberado
14:44:46 [INFO ] nucleo_libre    Nucleo 0 liberado
14:44:46 [INFO ] nucleo_libre    Nucleo 3 liberado
14:44:46 [INFO ] nucleo_libre    Nucleo 4 liberado
14:44:46 [INFO ] nucleo_libre    Nucleo 6 liberado
14:44:49 [AVISO ] proceso_pesado  System Idle Process consume 247.7% de CPU
14:44:55 [INFO ] proceso_cerrado  Proceso terminado: MoUsoCoreWorker.exe
14:44:55 [AVISO ] proceso_pesado  System Idle Process consume 335.1% de CPU
14:44:55 [INFO ] reporte      Reporte guardado en bitacora.json (6 metricas)
14:45:00 [AVISO ] proceso_pesado  System Idle Process consume 407.8% de CPU
14:45:02 [AVISO ] nucleo_saturado  Nucleo 0 saturado (92.2%)
14:45:03 [INFO ] nucleo_libre    Nucleo 0 liberado
14:45:05 [AVISO ] proceso_pesado  System Idle Process consume 248.1% de CPU
```

Figura 9. Eventos provocados por la carga de CPU, los núcleos saturados y el swap.

La captura muestra la secuencia de eventos emitidos por el despachador: primero el flanco swap_activo, luego los avisos nucleo_saturado de los ocho núcleos con su porcentaje, la alerta cpu_alta al superar el umbral de 80 % (con histéresis hasta 50 %), los avisos proceso_pesado con el consumo de cada proceso, los eventos INFO nucleo_libre cuando la carga baja y, por tiempo, el reporte guardado en bitacora.json.

7.2 Evento por falla o ausencia

Para demostrar el flujo de falla se simuló la ausencia de lectura en un sensor. El detector trata None como sensor_ausente y el despachador registra el evento como FALLA sin detener el monitoreo; además encola la alerta sonora zumbido_falla en el hilo descrito en la sección 5.

```
FALLA simulada: se fuerza una lectura None para demostrar sensor_ausente.
```

Figura 10. Evento de falla o ausencia de un sensor.

Salida del evento simulado: una lectura None se interpreta como sensor_ausente, se registra con nivel FALLA y el ciclo continúa con el resto de las métricas. El sistema no se detiene ante la falta de un sensor.

7.3 Reporte por tiempo y bitácora JSON

El evento de reporte se genera por tiempo o con el botón Generar reporte. El sistema resume las muestras del periodo, escribe bitacora.json y limpia el historial sin borrar la ventana móvil.

```
[
  {
    "nodo": "laptop-QuerubeAriza",
    "ubicacion": "Querube Ariza - Laboratorio 3 FISC",
    "hora": "2026-09-16 14:39:26",
    "metricas": {
      "cpu": {
        "n": 1,
        "min": 24.3,
        "max": 24.3,
        "prom": 24.3
      },
      "memoria": {
        "n": 1,
        "min": 67.8,
        "max": 67.8,
        "prom": 67.8
      },
      ... salida recortada para el informe ...
    },
    "memoria": {
      "n": 1,
      "min": 67.7,
      "max": 67.7,
      "prom": 67.7
    },
    "red": {
      "n": 1,
      "min": 259.9,
      "max": 259.9,
      "prom": 259.9
    }
  }
]
```

Figura 11. Resumen del periodo guardado en bitacora.json.

Fragmento del archivo bitacora.json: conserva la marca de tiempo del reporte, el nodo y el resumen de cada métrica del periodo (mínimo, máximo y promedio). Ilustra la estrategia medir, acumular, resumir y vaciar propia de los sistemas IoT.

8. Pruebas unitarias

El paquete pruebas contiene cuatro suites de pruebas unitarias con unittest, más una prueba en vivo. Las pruebas congelan el hilo reproductor (sonidos.detener()) y ponen el backend en silencio (SONIDO_BACKEND_NULO), de modo que corren sin emitir un solo tono.

Tabla 4. Suites de pruebas unitarias y número de pruebas.

Total: 41 pruebas unitarias en cuatro archivos, todas orientadas a comprobar que el sistema reacciona sin bloquearse.

Archivo de pruebas	N.º	Qué comprueba
pruebas/test_sonidos.py	14	Encolar no reproducir, tope de la cola, cadencia y vencimiento por marca de tiempo, y síntesis de los 17 patrones a un WAV válido.
pruebas/test_conexion_red.py	13	Flancos de pérdida y recuperación, rebotes, interfaces reales y virtuales, y el recordatorio periódico durante la caída.
pruebas/test_integracion.py	9	El ciclo despacha la pérdida de red y el recordatorio por tiempo, y las lecturas siguen llegando con el sonido activo.
pruebas/test_ventana.py	5	Insignias de red, botón de sonido y refresco de la ventana gráfica sin errores.

8.1 Resultado de la ejecución

Se ejecutó el comando completo de la suite con modo verboso:

```
C:\Users\Windows 11\Downloads\monitor_iot_QuerubeAriza> entorno\Scripts\python.exe -m unittest discover -s pruebas -t . -v
```

```

test_estado_indeterminado_no_se_arriesga
(pruebas.test_conexion_red.TestFlanco.test_estado_indeterminado_no_se_arriesga) ... ok
test_flanco_de_perdida (pruebas.test_conexion_red.TestFlanco.test_flanco_de_perdida) ... ok
test_flanco_de_recuperacion_informa_la_duracion
(pruebas.test_conexion_red.TestFlanco.test_flanco_de_recuperacion_informa_la_duracion) ... ok
test_primera_muestra_solo_fija_la_base
(pruebas.test_conexion_red.TestFlanco.test_primera_muestra_solo_fija_la_base) ... ok
test_rebote_no_produce_flanco (pruebas.test_conexion_red.TestFlanco.test_rebote_no_produce_flanco) ... ok
test_ignora_loopback_y_adaptadores_virtuales
(pruebas.test_conexion_red.TestInterfaces.test_ignora_loopback_y_adaptadores_virtuales) ... ok
test_interfaz_activa_sin_direccion_no_cuenta
(pruebas.test_conexion_red.TestInterfaces.test_interfaz_activa_sin_direccion_no_cuenta) ... ok
test_si_psutil_falla_no_hay_dato (pruebas.test_conexion_red.TestInterfaces.test_si_psutil_falla_no_hay_dato) ...
ok
test_sin_interfaces_no_hay_dato (pruebas.test_conexion_red.TestInterfaces.test_sin_interfaces_no_hay_dato) ...
ok
test_todas_las_interfaces_reales_caidas
(pruebas.test_conexion_red.TestInterfaces.test_todas_las_interfaces_reales_caidas) ... ok
test_al_recuperar_se_detiene_el_recordatorio
(pruebas.test_conexion_red.TestRecordatorio.test_al_recuperar_se_detiene_el_recordatorio) ... ok
test_el_recordatorio_se_repite_mientras_dure_la_caida
(pruebas.test_conexion_red.TestRecordatorio.test_el_recordatorio_se_repite_mientras_dure_la_caida) ... ok
test_recordatorio_solo_despues_del_periodo
(pruebas.test_conexion_red.TestRecordatorio.test_recordatorio_solo_despues_del_periodo) ... ok
test_al_recuperar_el_ciclo_lo_anuncia (pruebas.test_integracion.TestCiclo.test_al_recuperar_el_ciclo_lo_anuncia)
... ok
test_el_ciclo_despacha_la_perdida_de_red
(pruebas.test_integracion.TestCiclo.test_el_ciclo_despacha_la_perdida_de_red) ... ok
test_el_ciclo_dispara_el_recordatorio_por_tiempo
(pruebas.test_integracion.TestCiclo.test_el_ciclo_dispara_el_recordatorio_por_tiempo)
... salida recortada para el informe ...
test_apagado_no_encola (pruebas.test_sonidos.TestCola.test_apagado_no_encola) ... ok
test_cadencia_por_marca_de_tiempo (pruebas.test_sonidos.TestCola.test_cadencia_por_marca_de_tiempo) ... ok
test_encolar_es_inmediato_y_no_reproduce
(pruebas.test_sonidos.TestCola.test_encolar_es_inmediato_y_no_reproduce) ... ok
test_encolar_no_arranca_el_hilo (pruebas.test_sonidos.TestCola.test_encolar_no_arranca_el_hilo) ... ok
test_evento_sin_patron_no_suena (pruebas.test_sonidos.TestCola.test_evento_sin_patron_no_suena) ... ok
test_la_cola_no_supera_su_tope (pruebas.test_sonidos.TestCola.test_la_cola_no_supera_su_tope) ... ok
test_alerta_vencida_se_descarta (pruebas.test_sonidos.TestMarcasDeTiempo.test_alerta_vencida_se_descarta) ... ok
test_alerta_vigente_se_entrega (pruebas.test_sonidos.TestMarcasDeTiempo.test_alerta_vigente_se_entrega) ... ok
test_backend_en_silencio_no_lanza_error
(pruebas.test_sonidos.TestPatrones.test_backend_en_silencio_no_lanza_error) ... ok
test_cada_evento_apunta_a_un_patron_existente
(pruebas.test_sonidos.TestPatrones.test_cada_evento_apunta_a_un_patron_existente) ... ok
test_los_patrones_estan_bien_formados (pruebas.test_sonidos.TestPatrones.test_los_patrones_estan_bien_formados)
... ok
test_sintetiza_un_wav_valido (pruebas.test_sonidos.TestPatrones.test_sintetiza_un_wav_valido) ... ok
test_todos_los_patrones_sintetizan (pruebas.test_sonidos.TestPatrones.test_todos_los_patrones_sintetizan) ... ok
test_el_boton_de_sonido_altera_el_reproductor
(pruebas.test_ventana.TestVentana.test_el_boton_de_sonido_altera_el_reproductor) ... ok
test_insignia_de_red_caida (pruebas.test_ventana.TestVentana.test_insignia_de_red_caida) ... ok
test_insignia_de_red_conectada (pruebas.test_ventana.TestVentana.test_insignia_de_red_conectada) ... ok
test_insignia_de_sonido_sigue_al_reproductor
(pruebas.test_ventana.TestVentana.test_insignia_de_sonido_sigue_al_reproductor) ... ok
test_refrescar_pinta_sin_errores (pruebas.test_ventana.TestVentana.test_refrescar_pinta_sin_errores) ... ok

-----
Ran 41 tests in 9.784s

OK

```

Figura 12. Resultado final de las 41 pruebas unitarias (modo verboso).

Salida completa de la ejecución. Cada línea termina en ok, lo que indica que la prueba pasó; al final aparece el resumen del ejecutador: Ran 41 tests ... y la palabra OK, que confirma que

no hubo fallos ni errores. Las pruebas de la suite de sonidos verifican justamente que encolar es inmediato y que el hilo de audio trabaja aparte del ciclo de monitoreo.

8.2 Capturas de las pruebas unitarias

```
test_al_silenciar_se_vacia_laCola (pruebas.test_sonidos.TestCola.test_al_silenciar_se_vacia_laCola) ... ok
test_apagado_no_encola (pruebas.test_sonidos.TestCola.test_apagado_no_encola) ... ok
test_cadencia_por_marca_de_tiempo (pruebas.test_sonidos.TestCola.test_cadencia_por_marca_de_tiempo) ... ok
test_encolar_es_inmediato_y_no_reproduce (pruebas.test_sonidos.TestCola.test_encolar_es_inmediato_y_no_reproduce) ... ok
test_encolar_no_arranca_el_hilo (pruebas.test_sonidos.TestCola.test_encolar_no_arranca_el_hilo) ... ok
test_evento_sin_patron_no_suena (pruebas.test_sonidos.TestCola.test_evento_sin_patron_no_suena) ... ok
test_laCola_no_supera_su_tope (pruebas.test_sonidos.TestCola.test_laCola_no_supera_su_tope) ... ok
test_alerta_vencida_se_descarta (pruebas.test_sonidos.TestMarcasDeTiempo.test_alerta_vencida_se_descarta) ... ok
test_alerta_vigente_se_entrega (pruebas.test_sonidos.TestMarcasDeTiempo.test_alerta_vigente_se_entrega) ... ok
test_backend_en_silencio_no_lanza_error (pruebas.test_sonidos.TestPatrones.test_backend_en_silencio_no_lanza_error) ... ok
test_cada_evento_apunta_a_un_patron_existente (pruebas.test_sonidos.TestPatrones.test_cada_evento_apunta_a_un_patron_existente) ... ok
test_los_patrones_estan_bien_formados (pruebas.test_sonidos.TestPatrones.test_los_patrones_estan_bien_formados) ... ok
test_sintetiza_un_wav_valido (pruebas.test_sonidos.TestPatrones.test_sintetiza_un_wav_valido) ... ok
test_todos_los_patrones_sintetizan (pruebas.test_sonidos.TestPatrones.test_todos_los_patrones_sintetizan) ... ok

-----
Ran 14 tests in 0.270s

OK

C:\Users\Windows 11\Downloads\monitor_iot_QuerubeAriza>
```

Figura 13. Pruebas unitarias del reproductor de alertas sonoras.

Captura de la ejecución de `python -m unittest pruebas.test_sonidos -v`. Se observan las pruebas de la cola (encolar es inmediato y no reproduce, encolar no arranca el hilo, la cola no supera su tope, la cadencia por marca de tiempo, el apagado no encola y el silenciamiento vacía la cola), las de marcas de tiempo (alerta vigente se entrega y alerta vencida se descarta) y las de patrones (los 17 patrones están bien formados, todos se sintetizan, un evento sin patrón no suena y el WAV resultante es válido). Cierra con el resumen del ejecutador.

```
test_flanco_de_perdida (pruebas.test_conexion_red.TestFlanco.test_flanco_de_perdida) ... ok
test_flanco_de_recuperacion_informa_la_duracion (pruebas.test_conexion_red.TestFlanco.test_flanco_de_recuperacion_informa_la_duracion) ... ok
test_primera_muestra_solo_fija_la_base (pruebas.test_conexion_red.TestFlanco.test_primera_muestra_solo_fija_la_base) ... ok
test_rebote_no_produce_flanco (pruebas.test_conexion_red.TestFlanco.test_rebote_no_produce_flanco) ... ok
test_ignora_loopback_y_adaptadores_virtuales (pruebas.test_conexion_red.TestInterfaces.test_ignora_loopback_y_adaptadores_virtuales) ... ok
test_interfaz_activa_sin_direccion_no_cuenta (pruebas.test_conexion_red.TestInterfaces.test_interfaz_activa_sin_direccion_no_cuenta) ... ok
test_si_psutil_falla_no_hay_dato (pruebas.test_conexion_red.TestInterfaces.test_si_psutil_falla_no_hay_dato) ... ok
test_sin_interfaces_no_hay_dato (pruebas.test_conexion_red.TestInterfaces.test_sin_interfaces_no_hay_dato) ... ok
test_todas_las_interfaces_reales_caidas (pruebas.test_conexion_red.TestInterfaces.test_todas_las_interfaces_reales_caidas) ... ok
test_al_recuperar_se_detiene_el_recordatorio (pruebas.test_conexion_red.TestRecordatorio.test_al_recuperar_se_detiene_el_recordatorio) ... ok
test_el_recordatorio_se_repite_mientras_dure_la_caida (pruebas.test_conexion_red.TestRecordatorio.test_el_recordatorio_se_repite_mientras_dure_la_caida) ... ok
test_recordatorio_solo_despues_del_periodo (pruebas.test_conexion_red.TestRecordatorio.test_recordatorio_solo_despues_del_periodo) ... ok
test_al_recuperar_el_ciclo_lo_anuncia (pruebas.test_integracion.TestCiclo.test_al_recuperar_el_ciclo_lo_anuncia) ... ok
test_el_ciclo_despacha_la_perdida_de_red (pruebas.test_integracion.TestCiclo.test_el_ciclo_despacha_la_perdida_de_red) ... ok
test_el_ciclo_dispara_el_recordatorio_por_tiempo (pruebas.test_integracion.TestCiclo.test_el_ciclo_dispara_el_recordatorio_por_tiempo) ... ok
El aviso no lo dispara un sensor: lo dispara el reloj. ... ok
test_las_lecturas_siguen_llegando_con_el_sonido_activo (pruebas.test_integracion.TestCiclo.test_las_lecturas_siguen_llegando_con_el_sonido_activo) ... ok
Encender el reproductor no cambia la forma del ciclo. ... ok
test_atender_encola_el_sonido_del_evento (pruebas.test_integracion.TestDespachador.test_atender_encola_el_sonido_del_evento) ... ok
test_los_eventos_de_conexion_tienen_manejador (pruebas.test_integracion.TestDespachador.test_los_eventos_de_conexion_tienen_manejador) ... ok
test_sin_manejador_no_suena (pruebas.test_integracion.TestDespachador.test_sin_manejador_no_suena) ... ok
test_todos_los_eventos_con_sonido_tienen_manejador (pruebas.test_integracion.TestDespachador.test_todos_los_eventos_con_sonido_tienen_manejador) ... ok
test_niveles_esperados (pruebas.test_integracion.TestManejadoresDeConexion.test_niveles_esperados) ... ok
test_al_silenciar_se_vacia_laCola (pruebas.test_sonidos.TestCola.test_al_silenciar_se_vacia_laCola) ... ok
test_apagado_no_encola (pruebas.test_sonidos.TestCola.test_apagado_no_encola) ... ok
test_cadencia_por_marca_de_tiempo (pruebas.test_sonidos.TestCola.test_cadencia_por_marca_de_tiempo) ... ok
test_encolar_es_inmediato_y_no_reproduce (pruebas.test_sonidos.TestCola.test_encolar_es_inmediato_y_no_reproduce) ... ok
test_encolar_no_arranca_el_hilo (pruebas.test_sonidos.TestCola.test_encolar_no_arranca_el_hilo) ... ok
test_evento_sin_patron_no_suena (pruebas.test_sonidos.TestCola.test_evento_sin_patron_no_suena) ... ok
test_laCola_no_supera_su_tope (pruebas.test_sonidos.TestCola.test_laCola_no_supera_su_tope) ... ok
test_alerta_vencida_se_descarta (pruebas.test_sonidos.TestMarcasDeTiempo.test_alerta_vencida_se_descarta) ... ok
test_alerta_vigente_se_entrega (pruebas.test_sonidos.TestMarcasDeTiempo.test_alerta_vigente_se_entrega) ... ok
test_backend_en_silencio_no_lanza_error (pruebas.test_sonidos.TestPatrones.test_backend_en_silencio_no_lanza_error) ... ok
test_cada_evento_apunta_a_un_patron_existente (pruebas.test_sonidos.TestPatrones.test_cada_evento_apunta_a_un_patron_existente) ... ok
test_los_patrones_estan_bien_formados (pruebas.test_sonidos.TestPatrones.test_los_patrones_estan_bien_formados) ... ok
test_sintetiza_un_wav_valido (pruebas.test_sonidos.TestPatrones.test_sintetiza_un_wav_valido) ... ok
test_todos_los_patrones_sintetizan (pruebas.test_sonidos.TestPatrones.test_todos_los_patrones_sintetizan) ... ok
test_el boton_de_sonido_altera_el_reproductor (pruebas.test_ventana.TestVentana.test_el boton_de_sonido_altera_el_reproductor) ... ok
test_insignia_de_red_caida (pruebas.test_ventana.TestVentana.test_insignia_de_red_caida) ... ok
```

Figura 14. Pruebas unitarias de la suite completa del proyecto.

Captura de la ejecución de `python -m unittest discover -s pruebas -t . -v`. Se muestra la lista de pruebas de las cuatro suites: conexión de red (flancos, interfaces y recordatorio), integración (el ciclo despacha eventos y sigue midiendo con el sonido activo), sonidos (cola, cadencia y patrones) y ventana (insignias, botón de sonido y refresco). Todas aparecen marcadas como ok.

9. Conclusión

- La computadora funciona como un nodo IoT porque entrega mediciones reales del sistema operativo y permite reaccionar ante cambios de su estado.
- La organización en paquetes facilita mantener responsabilidades separadas: sensores, eventos, almacenamiento, núcleo, dashboard y reproductor de alertas.
- La ventana móvil con deque(maxlen=N) reduce falsos positivos, ya que los umbrales se evalúan sobre promedios y no solo sobre lecturas instantáneas.
- El uso de un despachador basado en diccionario evita cadenas extensas de if/elif y permite agregar eventos nuevos con menor riesgo de modificar el bucle principal.
- El registro JSON permite conservar resúmenes del periodo y demuestra la estrategia de medir, acumular, resumir y vaciar usada en sistemas IoT.
- La versión web reutiliza el mismo núcleo de monitoreo, lo que confirma que al separar el motor de la presentación se puede cambiar la interfaz sin modificar los sensores ni la detección de eventos.
- El subsistema de alertas sonoras demuestra que un sonido no tiene por qué bloquear el sistema: quien detecta solo encola y quien suena es un hilo aparte, por lo que el ciclo de monitoreo nunca espera audio.
- Los controles por marca de tiempo (cadencia, vencimiento y cola acotada) son suficientes para que una ráfaga de eventos no se convierta en una ráfaga de sonidos ni en una cola indefinida.
- El diseño sonoro por familias (graves para almacenamiento, agudos para CPU, barridos para red, cuadrada con ruido para fallas y una única campanita para las recuperaciones) permite identificar el problema solo con escuchar, sin mirar la pantalla.
- Las 41 pruebas unitarias pasaron sin fallos, y entre ellas están las que comprueban que encolar es inmediato y que las lecturas siguen llegando con el reproductor encendido: la condición del laboratorio queda verificada de forma automatizada.